

EST Dynamic Form Guide

One-step vs Multi-step — Step-by-step with a simple example

Use the same DynamicForm engine for both cases. One-step = one fill page. Multi-step = several fill pages chained by MDMS nextStep, then one Check and one Submit.

Rule: Do not invent a second form system. Reuse DynamicForm → DynamicFormStep → DynamicCheckPage + MDMS body[] steps.

1. Quick comparison

Topic	One-step (today)	Multi-step
Fill UI	1× DynamicFormStep	N× DynamicFormStep
Config	1 MDMS step + form[]	N MDMS steps, each with form[] + nextStep
Session	One step key	One key per step, all in session
Check	Summarize that step	Summarize all steps
Navigation	Save & Next → /check	Save & Next → next route → ... → /check

2. Flow A — One-step (current EST)

Registration and Assign Assets today mostly use one fill page.



Examples

- Registration: /newRegistration → /check → /acknowledgement
- Assign assets: /assign-assets → /check → /acknowledgement

3. Flow B — Multi-step (step by step)

Follow these steps when one form page is not enough.

Step 1 — Split MDMS into ordered steps

In MDMS (e.g. Estate.NewRegistration), add multiple body entries. Each step owns its own form[]. Last step uses nextStep: null → /check.

```
body: [
  { route: "asset-basic", key: "assetBasic",
    component: "ESTAssetBasic", nextStep: "owner-details",
    form: [ /* asset fields */ ] },

  { route: "owner-details", key: "ownerDetails",
    component: "ESTOwnerDetails", nextStep: "documents",
    form: [ /* owner fields */ ] },

  { route: "documents", key: "documents",
    component: "ESTDocuments", nextStep: null,    // → /check
    form: [ /* file fields */ ] }
]
```

Step 2 — Thin React wrapper per step

Each route component is a thin shell around `DynamicFormStep`. Register it like `ESTNEWRegistration`.

```
// PageComponents/ESTAssetBasic.js
const ESTAssetBasic = ({ onSelect, config, formData, t }) => (
  <DynamicFormStep
    config={config}
    localOverrides={assetBasicOverrides}
    onSelect={onSelect}
    formData={formData}
    t={t}
    confirmCancel
  />
);
```

Step 3 — Wire wizard index

- Create/index.js already maps `config.map` → Route.
- Keep `useEstWizard({ multiStepNavigation: true })`.
- Add every step route to `terminalSegments` plus check and acknowledgement.

Step 4 — Local overrides (JS only)

Put labels, types, order, visibleWhen, dropdown masters in MDMS. Put `staticFields`, `computedFields`, `crossFieldValidations` in local overrides.

Step 5 — Check page + one Submit

- Session stores each step via `mergeSessionStepWithRouteConfig`.
- Check page reads all step keys / `routeConfigs` and builds summary sections.
- One Submit builds the final API payload from the full session.

```
asset-basic → owner-details → documents → /check → API → /acknowledgement
  ■           ■           ■
  ■■■■■■■■■ session (all steps + routeConfigs) ■■■■■■■■■
```

4. Simple example — Citizen complaint (3 steps)

Imagine a tiny multi-step wizard: Location → Details → Documents. Same pattern as EST; only field names differ.

4.1 User journey

What the user sees

- 1) **Location** — locality + address → Save & Next
- 2) **Details** — category + description → Save & Next
- 3) **Documents** — photo upload → Save & Next
- 4) **Check** — review all three sections → Submit
- 5) **Acknowledgement** — application number

4.2 MDMS (simplified)

```
{
  "tenantId": "pg",
  "moduleName": "Estate",
  "ComplaintWizard": [{
    "head": "COMPLAINT_CREATE",
    "body": [
      {
        "route": "location",
        "key": "location",
        "component": "CMPLocation",
        "pageHeading": "CMP_LOCATION",
        "nextStep": "details",
        "payloadKey": "Complaints",
        "form": [
          { "name": "locality", "type": "dropdown", "label": "CMP_LOCALITY",
            "order": 1, "validation": { "required": true },
            "dataSource": { "type": "mdms", "module": "egov-location", "master": "TenantBoundary" } },
          { "name": "address", "type": "text", "label": "CMP_ADDRESS",
            "order": 2, "validation": { "required": true, "minLength": 5 } }
        ]
      },
      {
        "route": "details",
        "key": "details",
        "component": "CMPDetails",
        "pageHeading": "CMP_DETAILS",
        "nextStep": "documents",
        "form": [
          { "name": "category", "type": "dropdown", "label": "CMP_CATEGORY",
            "order": 1, "validation": { "required": true } },
          { "name": "description", "type": "textarea", "label": "CMP_DESC",
            "order": 2, "validation": { "required": true, "maxLength": 500 } }
        ]
      },
      {
        "route": "documents",
        "key": "documents",
        "component": "CMPDocuments",
        "pageHeading": "CMP_DOCUMENTS",
        "nextStep": null,
        "form": [
          { "name": "photo", "type": "file", "label": "CMP_PHOTO",
            "order": 1, "validation": { "required": false } }
        ]
      }
    ]
  }]
}
```

4.3 One step wrapper (same for all three)

```
import { DynamicFormStep } from "@nudmcdgnpm/digit-ui-react-components";

const CMPLocation = ({ onSelect, config, formData, t }) => (
  <DynamicFormStep
    config={config}
    localOverrides={{}} // add crossFieldValidations only if needed
    onSelect={onSelect}
    formData={formData}
    t={t}
    confirmCancel
  />
);

// Register: Digit.ComponentRegistryService.setComponent("CMPLocation", CMPLocation);
// Same pattern for CMPDetails and CMPDocuments.
```

4.4 Wizard shell (index)

```
const { config, handleSelect, onCheckSuccess, isReady } = useEstWizard({
  mdmsData: complaintConfig,
  isLoading,
  indexRoute: "location",
  sessionKey: "CMP_CREATE",
  terminalSegments: ["location", "details", "documents", "check", "acknowledgement"],
  multiStepNavigation: true,
});

// Routes: config.map(step => <Route path={step.route} element={<Step ... /> />)
// Plus /check and /acknowledgement
```

4.5 Session after each Save & Next

```
// After step 1
{ location: { Complaints: [{ locality: {...}, address: "..."} ], __routeConfig: {...} },
  routeConfigs: { location: {...} } }

// After step 2 (merged)
{ location: {...},
  details: { category: {...}, description: "..."},
  routeConfigs: { location: {...}, details: {...} } }

// After step 3 → navigate to /check → Submit builds one API payload from all keys
```

5. Global functions needed in BOTH scenarios

These shared helpers apply whether you have 1 or N form steps. Do not re-implement in the module.

Area	Functions	File
Config	mergeRouteConfig, flattenFormConfig, sortByOrder, isFieldVisible, findFieldConfig	formUtils / checkPageUtils
Validate	validateFields, validateCrossField	validators.js
Prefill / API	buildInitialData, buildPayload, buildApiPayload, dropdown helpers	formUtils / payloadUtils
Session / Check	attachRouteConfigToStepData, mergeSessionStepWithRouteConfig, buildSummarySections	checkPageUtils.js
Wizard shell	useEstWizard, buildWizardSteps, createWizardGoNext	EST utils

Multi-step only extras

- MDMS nextStep chain (and optional skip)
- Per-step wrappers + registry entries

- Check page summarizing N steps
- Final payload merges all step flats on Submit
- Optional progress / timeline UI driven by wizard config

6. Checklist when adding multi-step

#	Action	Done?
1	Add MDMS steps with route, key, form, nextStep, component	■
2	Add thin DynamicFormStep wrappers + registry	■
3	Local overrides only for JS behavior	■
4	Update terminalSegments / indexRoute	■
5	Teach check + payload builder every step key	■
6	Keep Cancel / draft / confirmCancel opt-in per step	■

Memory tip: DynamicForm = one page of fields. Multi-step = several pages of DynamicForm, chained by MDMS nextStep, one shared Check + Submit.

Companion docs: [DYNAMIC_FORM_FLOW.md](#) · [MULTI_STEP_FORM_FLOW.md](#)